Machine Learning Workshop

Customer Behavioral Analysis in Retail

REPORT

Practical Workshop: Gift E-commerce

Prepared by: Beya Sayah

Année Univeristaire : 2025-2026

Contents

1 Problem Context and Objective **3**
1.1 What is Customer Churn? . . . 3
1.2 Why Preprocessing is Critical for Churn Prediction . . . 3

2 Churn classification Model:Step 1 — Initial Data **3**
2.1 What We Did . . . 3

3 Step 2 — Type Conversion **4**
3.1 The Problem . . . 4
3.2 The Fix . . . 4

4 Step 3 — Outlier and Error Code Correction **4**
4.1 The Problem . . . 4
4.2 The Fix . . . 5

5 Step 4 — Date Parsing and Temporal Feature Extraction **5**
5.1 What We Did . . . 5

6 Step 5 — IP Address Geolocation Feature Engineering **5**
6.1 What We Did . . . 5
6.2 The CountryMismatch Feature . . . 6

7 Step 6 — Dropping Irrelevant **6**
7.1 Columns Removed and Rationale . . . 7

8 Step 7 — Feature Engineering from Raw Aggregates **7**
8.1 What We Did . . . 7
8.2 Why These Features Matter for Churn . . . 8
8.3 Dropping the Source Columns . . . 8

9 Step 8 — Data Leakage Removal **9**
9.1 The Problem — What is Data Leakage? . . . 9
9.2 Leaky Columns Identified and Removed . . . 9

10 Step 12 — Feature Selection and Leakage Verification **10**
10.1 Mutual Information Analysis . . . 10
10.2 TenureRatio — Late Leakage Detection . . . 11

11 Step 9 — Train/Test Split **11**
11.1 Why Split Before Any Further Transformation . . . 11
11.2 Key Design Decisions . . . 12

12 Step 10 — Missing Value Imputation **12**
12.1 Strategy Overview . . . 12
12.2 Median Imputation . . . 12
12.3 KNN Imputation for Age . . . 13
12.4 AgeCategory Re-derivation . . . 13

12.5 CountryMismatch Recomputation After Imputation . . . 14

13 Step 11 — Categorical Encoding 14
13.1 Encoding Strategy by Feature Type . . . 14
13.2 Ordinal Encoding . . . 15
13.3 One-Hot Encoding . . . 15
13.4 Target Encoding for High-Cardinality Geographies . . . 15

14 Summary: Full Preprocessing Pipeline 16
14.1 Final Dataset Dimensions . . . 16

15 Negative Behaviour Clustering 16

16 Tree-Based Model Training and Evaluation 20
16.1 Decision Tree . . . 20
16.1.1 Stump Test — Isolating the Dominant Feature . . . 21
16.2 Random Forest . . . 22
16.3 XGBoost . . . 22
16.3.1 Hyperparameter Tuning . . . 22
16.3.2 Feature Importance and Zero-Importance Removal . . . 23
16.3.3 Further Pruning Attempt — Stopped by Test Accuracy . . . 24
16.4 Final Results Summary . . . 24

17 Regression model:Data Preprocessing Pipeline 25
17.1 Model Overview . . . 25
17.2 Feature Selection: What We Dropped and Why . . . 25
17.3 Feature Engineering . . . 26
17.4 Data Cleaning . . . 26
17.5 Outlier Handling . . . 27
17.6 Train / Test Split . . . 27
17.7 Imputation . . . 27
17.8 Correlation Analysis . . . 27
17.9 Variance Inflation Factor (VIF) . . . 28
17.10 Categorical Encoding . . . 28
17.11 Feature Scaling . . . 29
17.12 Feature Variance . . . 29
17.13 PCA Visualisation . . . 30
17.13.1 Principal Component Loadings . . . 30

18 Marketing Applications of the Spending Prediction 32

19 Conclusion 32

1 Problem Context and Objective

1.1 What is Customer Churn?

Customer churn refers to the phenomenon where customers stop doing business with a company. In retail e-commerce, a customer is considered **churned** when they have not made a purchase over a significant period, or when behavioral signals indicate disengagement. Identifying churners before they leave allows the business to deploy targeted retention campaigns, personalized offers, and proactive customer service interventions.

1.2 Why Preprocessing is Critical for Churn Prediction

Raw transactional and CRM data collected from retail systems is almost never ready for machine learning. It typically contains:

- **Type inconsistencies:** numerical features stored as strings due to database export formats.
- **Error codes disguised as values:** sentinel values such as `-1`, `999`, or `0` injected by legacy systems.
- **Missing data:** incomplete customer profiles, especially for sensitive fields like age.
- **Data leakage:** features that encode the target variable directly, which would cause artificially inflated model performance and failure in production.
- **Redundant features:** correlated or constant columns that add noise without predictive value.

Each step in our preprocessing pipeline was designed to address one or more of these issues specifically in the context of churn prediction.

2 Churn classification Model:Step 1 — Initial Data

2.1 What We Did

The first step was to load the cleaned starting dataset and perform a structural audit before any transformation. We inspected column types, missing values, and duplicates:

Listing 1: Initial data audit

```
print("Shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum()[df.isnull().sum() > 0])
print("\nDuplicates:", df.duplicated().sum())
print("\nTypes:")
print(df.dtypes.value_counts())
```

3 Step 2 — Type Conversion

3.1 The Problem

Several monetary and behavioral columns were stored as `object` (string) dtype despite containing numerical data. This occurs when a CSV export from a database contains locale-specific formatting (commas as decimal separators) or mixed-type entries that caused pandas to default to string inference.

The affected columns were:

Column	Domain
`MonetaryTotal`, `MonetaryAvg`, `MonetaryStd`	Spending aggregates
`MonetaryMin`, `MonetaryMax`	Spending range
`AvgQuantityPerTransaction`	Purchase behavior
`AvgDaysBetweenPurchases`	Recency proxy
`AvgProductsPerTransaction`	Basket composition
`AvgLinesPerInvoice`	Invoice complexity

3.2 The Fix

Listing 2: Numeric type conversion

```
cols_to_convert = [
    'MonetaryTotal', 'MonetaryAvg', 'MonetaryStd', 'MonetaryMin', '
   MonetaryMax',
    'AvgQuantityPerTransaction', 'AvgDaysBetweenPurchases',
    'AvgProductsPerTransaction', 'AvgLinesPerInvoice'
]
for col in cols_to_convert:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

The `errors='coerce'` parameter converts any unparseable values to `NaN` instead of raising an exception. This is intentional — it turns hidden data quality issues into explicit missing values that can be handled systematically in the imputation step.

Why This Matters for Churn

Monetary features are among the strongest churn predictors. A customer whose spending is declining (`MonetaryTotal` decreasing over time) is a strong churn candidate. If these columns remain as strings, no model can learn this relationship.

4 Step 3 — Outlier and Error Code Correction

4.1 The Problem

Two columns contained **sentinel values** — special codes inserted by upstream systems to represent errors, missing data, or system states — that are not valid domain values:

- `SupportTicketsCount`: Values of `-1` and `999` are error codes from the ticketing system. A negative ticket count is physically impossible; `999` is a placeholder for system errors.

- `SatisfactionScore`: The valid scale is 1–5. Values of `-1`, `0`, and `99` indicate survey non-responses or system injection errors.

4.2 The Fix

Listing 3: Sentinel value replacement

```
df['SupportTicketsCount'] = df['SupportTicketsCount'].replace([-1, 999], np.nan)
df['SatisfactionScore'] = df['SatisfactionScore'].apply(
    lambda x: np.nan if x in [-1, 0, 99] else x
)
```

We replace these values with `NaN` rather than a fixed number so they are treated as missing data in subsequent imputation — not as valid observations that could mislead the model.

Common Mistake to Avoid

Replacing `999` with `0` would be wrong — it would tell the model that this customer had zero support tickets, which is a completely different behavioral signal from "we don't know how many tickets this customer had."

5 Step 4 — Date Parsing and Temporal Feature Extraction

5.1 What We Did

The `RegistrationDate` column was stored as a string. We parsed it into a proper datetime object and extracted two numerical features from it:

Listing 4: Date parsing and extraction

```
df['RegistrationDate'] = pd.to_datetime(df['RegistrationDate'],
                                        dayfirst=True, errors='coerce')
df['RegYear']  = df['RegistrationDate'].dt.year
df['RegMonth'] = df['RegistrationDate'].dt.month
df.drop(columns=['RegistrationDate'], inplace=True)
```

6 Step 5 — IP Address Geolocation Feature Engineering

6.1 What We Did

The raw `LastLoginIP` column contains IP addresses — strings with no direct predictive value. We used the GeoLite2 database to resolve each IP to a country, creating a new feature `GeoIP`:

Listing 5: IP geolocation

```
import geoip2.database
reader = geoip2.database.Reader('GeoLite2-Country.mmdb')

def get_country(ip):
    if pd.isna(ip) or not isinstance(ip, str):
        return np.nan
    try:
        return reader.country(ip).country.name
    except:
        return np.nan

df['GeoIP'] = df['LastLoginIP'].apply(get_country)
```

6.2 The CountryMismatch Feature

Beyond geolocation, we engineered a new binary feature `CountryMismatch` by comparing the customer's registered `Country` against the country resolved from their last login IP:

Listing 6: CountryMismatch feature

```
df['CountryMismatch'] = np.where(
    df['Country'].isna() | df['GeoIP'].isna(),
    np.nan,
    (df['Country'] != df['GeoIP']).astype(int)
)
```

Business Rationale for CountryMismatch

A customer logging in from a country different from their registered country could indicate: (1) account sharing or fraud, (2) travel, or (3) VPN usage. All three scenarios have different churn implications. This feature encodes a behavioral anomaly that raw geographic data alone cannot capture.

After extraction, the raw `LastLoginIP` column was dropped — it has no further value once the signal has been extracted.

7 Step 6 — Dropping Irrelevant

7.1 Columns Removed and Rationale

Column	Reason for Removal
`NewsletterSubscribed`	Zero-variance feature — all customers had the value "Yes". A constant feature provides no discriminative power to any model.
`UniqueCountries`	Low variation, near constant
`CustomerID`	A unique identifier per customer. Including it would allow the model to memorize individual customers rather than learning generalizable patterns.
`Gender`	Ethical exclusion. Differentiating churn-based marketing interventions by gender raises fairness concerns and may constitute discriminatory practice under data protection regulations.

8 Step 7 — Feature Engineering from Raw Aggregates

8.1 What We Did

Five new features were derived by combining existing raw aggregates into more semantically meaningful ratios:

Listing 7: Derived ratio features

```
df['MonetaryPerDay']    = df['MonetaryTotal'] / (df['CustomerTenureDays'] + 1)
df['AvgBasketValue']    = df['MonetaryTotal'] / (df['Frequency'] + 1)
df['TenureRatio']       = df['Recency'] / (df['CustomerTenureDays'] + 1)
df['CancelRate']        = df['CancelledTransactions'] / (df['TotalTransactions'] + 1)
df['ProductsPerTrans']  = df['UniqueProducts'] / (df['TotalTransactions'] + 1)
```

Note the `+1` in all denominators — this prevents division by zero for customers with no transactions or zero tenure.

8.2 Why These Features Matter for Churn

Feature	Churn Interpretation
`MonetaryPerDay`	Revenue intensity normalized by tenure. A customer spending less per day recently than historically is decelerating — a churn signal.
`AvgBasketValue`	Average spend per visit. Declining basket value over time precedes churn.
`TenureRatio`	How recently the customer was last active relative to how long they have been a customer. A high ratio (inactive for most of their tenure) is a strong churn indicator.
`CancelRate`	Proportion of transactions that were cancelled. High cancellation rates reflect dissatisfaction, a leading churn predictor.
`ProductsPerTrans`	Breadth of product engagement. Customers exploring many product categories are more engaged and less likely to churn.

8.3 Dropping the Source Columns

After creating the ratios, the raw source columns were dropped:

Listing 8: Source column removal after engineering

```
df = df.drop(columns=[
    'Recency', 'CustomerTenureDays',
    'CancelledTransactions', 'UniqueProducts'
])
```

Special Case: `Recency` is a Leakage Feature

Among the dropped columns, `Recency` deserves particular attention. Unlike the others which are dropped purely to avoid redundancy, `Recency` is a **direct definition of churn**: it measures how many days have passed since the customer's last purchase. A high recency value *is* churn by another name — the model would not be learning to *predict* churn, it would be reading it directly from the data.

This avoids **multicollinearity**: keeping both the source columns and their derived ratios would make the model see the same information twice, potentially distorting feature importance.

A similar leakage issue was encountered with a feature engineered during exploration:

Listing 9: Leaky engineered feature

```
df['RecencyVsAvg'] = df['Recency'] - df['AvgDaysBetweenPurchases']
```

The intuition behind `RecencyVsAvg` was sound: comparing how long a customer has been inactive relative to their typical purchase rhythm should capture whether silence is unusual for that specific customer. A customer who normally buys every 5 days being

absent for 60 days is a much stronger churn signal than a customer who normally buys every 90 days.

Why `RecencyVsAvg` Was Also Removed

Despite the valid business logic, this feature inherits the leakage problem from `Recency`. Since `Recency` directly encodes churn, any linear combination or transformation of it — including subtraction — carries the same leakage. The formula reduces to:

$$\texttt{RecencyVsAvg} = \underbrace{\texttt{Recency}}_{\text{leaky}} - \texttt{AvgDaysBetweenPurchases}$$

9 Step 8 — Data Leakage Removal

9.1 The Problem — What is Data Leakage?

Data leakage occurs when features in the training set contain information that **directly encodes the target variable** or information that would not be available at prediction time in production. A model trained on leaky features achieves artificially high performance during evaluation but fails completely when deployed.

9.2 Leaky Columns Identified and Removed

Column	Why It Leaks
`ChurnRiskCategory`	Directly derived from the churn label. Categories like "High Risk" are post-hoc classifications based on `Churn=1`. Using this feature is equivalent to giving the model the answer.
`CustomerType`	Contains values like "Perdu" (Lost), which directly identifies churned customers.
`AccountStatus`	Contains values like "Suspended", which is a consequence of churn, not an independent predictor.
`RFMSegment`	RFM segmentation assigns labels like "Dormant" which map almost perfectly to churn.

Listing 10: Leakage removal

```
df = df.drop(columns=[
    'ChurnRiskCategory', 'CustomerType', 'AccountStatus','
  RFMSegment'])
```

Critical: Why Leakage is the Most Dangerous Bug

A model trained on `ChurnRiskCategory` might achieve 99% accuracy in evaluation. But in production, this column does not exist at prediction time (it is only assigned *after* a customer churns). The model would have learned nothing real about churn behavior.

10 Step 12 — Feature Selection and Leakage Verification

10.1 Mutual Information Analysis

After encoding, we computed the mutual information between each feature and the churn target. Mutual information measures the statistical dependency between a feature and the target, regardless of the relationship being linear or nonlinear:

Listing 11: Mutual information feature selection

```
from sklearn.feature_selection import mutual_info_classif

mi = pd.Series(
    mutual_info_classif(X_train, y_train, random_state=42),
    index=X_train.columns
).sort_values(ascending=False)
```

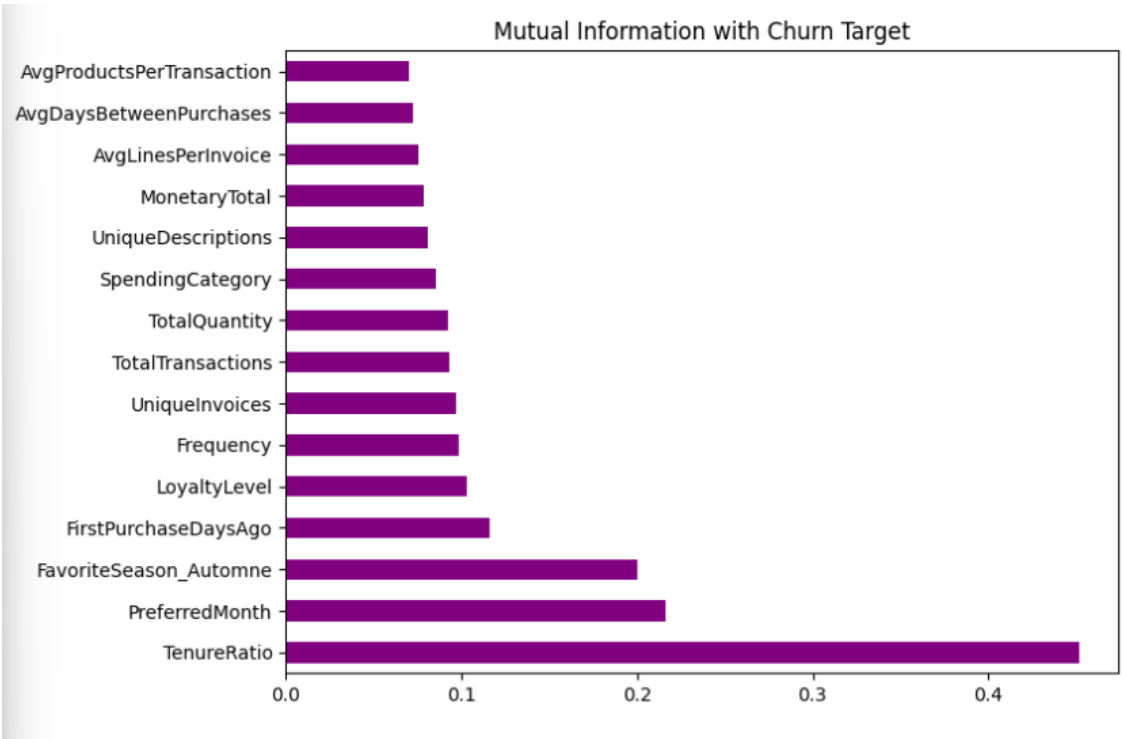


Figure 1: mutual information with churn target

10.2 TenureRatio — Late Leakage Detection

The mutual information analysis revealed that `TenureRatio` had anomalously high dependency with the churn target. A box plot confirmed the issue:

Listing 12: TenureRatio leakage verification

```
sns.boxplot(x=y_train, y=X_train['TenureRatio'])
plt.title('Tenure Ratio vs Churn (Train set)')
```

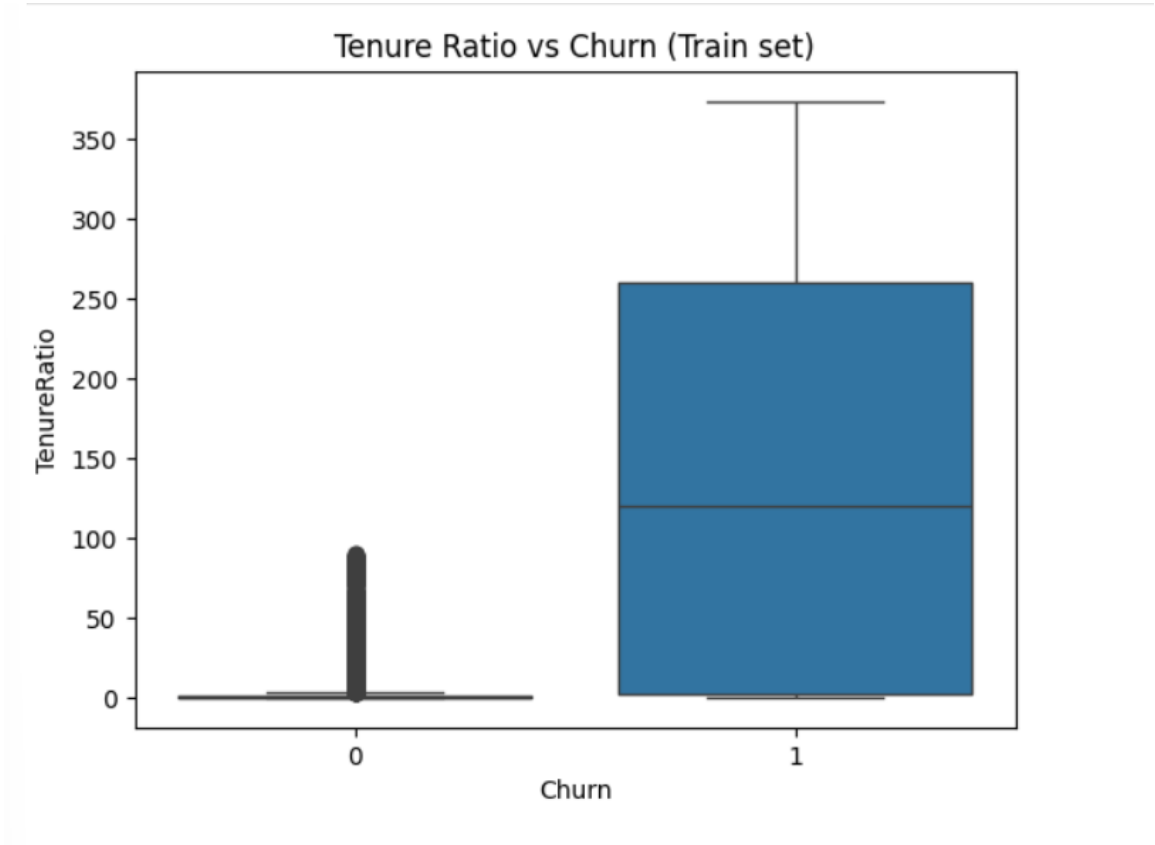


Figure 2: Tenure Ratio vs Churn — the clean separation between classes confirms this feature directly encodes the target.

`TenureRatio` = `Recency` / `CustomerTenureDays` is essentially a re-definition of churn: a customer who has been inactive for most of their tenure *is*, by definition, a churner. This feature would have caused near-perfect training performance followed by failure in production. It was removed.

11 Step 9 — Train/Test Split

11.1 Why Split Before Any Further Transformation

The train/test split is performed **before imputation, encoding, and scaling**. This is a fundamental requirement of sound machine learning practice:

Listing 13: Stratified train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

11.2 Key Design Decisions

Stratified split (`stratify=y`): Churn datasets are inherently imbalanced — churners are a minority class. Without stratification, the test set might contain a different proportion of churners than the training set, making evaluation metrics unreliable. Stratification preserves the original class ratio in both splits.

80/20 ratio: With a retail dataset of moderate size, 80% training data provides enough examples of both churners and non-churners for the models to learn meaningful patterns.

Fit on train, transform both: All subsequent transformers (imputers, encoders, scalers) are `fit()` exclusively on `X_train` and `transform()` applied to both sets. Fitting on the full dataset before splitting would cause **data leakage from the test set** — the imputer would know test set statistics, artificially improving test performance.

12 Step 10 — Missing Value Imputation

12.1 Strategy Overview

Different features required different imputation strategies based on their statistical properties and the nature of their missingness:

Strategy	Applied to	Rationale
Median imputation	`AvgDaysBetweenPurchases`, `SupportTicketsCount`, `SatisfactionScore`	These features are right-skewed with outliers. Mean imputation would be distorted by extreme values; median is robust.
KNN imputation	`Age`, `Frequency`, `MonetaryTotal`	Age has ∼30% missing. Global median would lose behavioral variation. KNN finds the 5 most similar customers and estimates age from their values.
Constant "Unknown"	`GeoIP`	Privacy-preserving. Missing IP may indicate VPN use or privacy tools — itself a behavioral signal.
Re-derivation	`AgeCategory`	Re-derived from imputed `Age` values using business-defined age brackets.

12.2 Median Imputation

Listing 14: Median imputation

```
cols_median = ['AvgDaysBetweenPurchases', 'SupportTicketsCount', '
    SatisfactionScore']
imputer_median = SimpleImputer(strategy='median')
X_train[cols_median] = imputer_median.fit_transform(X_train[
    cols_median])
X_test[cols_median]  = imputer_median.transform(X_test[cols_median
    ])
```

12.3 KNN Imputation for Age

Age required a more sophisticated approach because 30% of values were missing. A global median would replace all missing ages with the same value, erasing demographic variation. KNN imputation finds the `k=5` nearest neighbors in feature space and estimates missing values from their actual ages:

Listing 15: KNN imputation for Age

```
cols_knn = ['Age', 'Frequency', 'MonetaryTotal']
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train[cols_knn])
X_test_scaled  = scaler.transform(X_test[cols_knn])

imputer = KNNImputer(n_neighbors=5)
X_train_imputed = imputer.fit_transform(X_train_scaled)
X_test_imputed  = imputer.transform(X_test_scaled)

# Inverse-transform back to original scale
X_train[cols_knn] = scaler.inverse_transform(X_train_imputed)
X_test[cols_knn]  = scaler.inverse_transform(X_test_imputed)
```

Why Scale Before KNN?

KNN measures Euclidean distance between data points. Without scaling, `MonetaryTotal` (range: 0–50,000) would completely dominate `Age` (range: 18–80) when computing distances. The imputed age would be driven entirely by spending similarity, not behavioral similarity. Standardizing ensures all features contribute equally to the distance calculation.

12.4 AgeCategory Re-derivation

After imputing `Age`, `AgeCategory` was re-derived using business-defined brackets rather than imputing the categorical column directly:

Listing 16: AgeCategory re-derivation

```
def assign_age_category(age):
    if pd.isna(age):     return 'Inconnu'
    elif age < 25:       return '18-24'
    elif age < 35:       return '25-34'
    elif age < 45:       return '35-44'
```

```
    elif age < 55:       return '45-54'
    elif age < 65:       return '55-64'
    else:                return '65+'

X_train['AgeCategory'] = X_train['Age'].apply(assign_age_category)
X_test['AgeCategory']  = X_test['Age'].apply(assign_age_category)
X_train = X_train.drop(columns=['Age'])
X_test  = X_test.drop(columns=['Age'])
```

The raw `Age` column was then dropped. Tree-based models learn non-linear relationships naturally, so the categorical binning preserves interpretability without losing model quality.

12.5 CountryMismatch Recomputation After Imputation

After imputing `GeoIP` with "Unknown", the `CountryMismatch` feature was recomputed to reflect the now-complete data:

Listing 17: CountryMismatch recomputation

```
X_train['CountryMismatch'] = (X_train['Country'] != X_train['GeoIP'
    ]).astype(int)
X_test['CountryMismatch']  = (X_test['Country']  != X_test['GeoIP'
    ]).astype(int)
```

13 Step 11 — Categorical Encoding

13.1 Encoding Strategy by Feature Type

The choice of encoding method depends on whether the categorical variable has a natural order and on its cardinality (number of unique values):

Method	Applied to	Rationale
Ordinal Encoding	`AgeCategory`, `SpendingCategory`, `LoyaltyLevel`, `BasketSizeCategory`, `PreferredTimeOfDay`	These features have a meaningful order (e.g., Low < Medium < High). Ordinal encoding preserves this order as integer ranks.
One-Hot Encoding	`FavoriteSeason`, `WeekendPreference`, `ProductDiversity`, `Region`	No natural order. OHE creates a binary column per category, letting the model learn independent weights for each.
Target Encoding	`Country`, `GeoIP`	High cardinality (>50 countries). OHE would create 50+ sparse columns. Target encoding replaces each category with its mean churn rate (smoothed).

13.2 Ordinal Encoding

Listing 18: Ordinal encoding with defined orders

```
ordinal_orders = {
    'AgeCategory':        ['18-24', '25-34', '35-44', '45-54', '
   55-64', '65+'],
    'SpendingCategory':   ['Low', 'Medium', 'High', 'VIP'],
    'LoyaltyLevel':       ['Nouveau', 'Jeune', 'Etabli', 'Ancien'],
    'BasketSizeCategory': ['Petit', 'Moyen', 'Grand'],
    'PreferredTimeOfDay': ['Matin', 'Midi', 'Apres-midi', 'Soir'],
}
encoder_ordinal = OrdinalEncoder(
    categories=[ordinal_orders[c] for c in ordinal_orders],
    handle_unknown='use_encoded_value',
    unknown_value=-1
)
```

The `handle_unknown='use_encoded_value'` parameter assigns `-1` to any category seen in the test set but not in training. This makes the pipeline robust to new demographic or behavioral segments that appear after training.

13.3 One-Hot Encoding

Listing 19: One-hot encoding for nominal features

```
encoder_ohe = OneHotEncoder(
    sparse_output=False,
    handle_unknown='ignore',
    drop=None         # no drop for tree models
)
```

Why no column drop for trees? Linear models require dropping one dummy column per feature to avoid the dummy variable trap (perfect multicollinearity). Tree-based models are immune to multicollinearity — they select splits independently and are not affected by redundant binary columns. Keeping all dummies makes feature importance interpretation cleaner.

13.4 Target Encoding for High-Cardinality Geographies

Listing 20: Target encoding for Country and GeoIP

```
import category_encoders as ce
encoder_target = ce.TargetEncoder(
    cols=['Country', 'GeoIP'],
    smoothing=1.0
)
X_train[target_cols] = encoder_target.fit_transform(X_train[
   target_cols], y_train)
X_test[target_cols]  = encoder_target.transform(X_test[target_cols
   ])
```

The `smoothing=1.0` parameter applies shrinkage: rare countries with few customers are pulled toward the global mean churn rate, preventing the model from over-fitting on small samples from minor geographies.

Target Encoding Leakage Risk

Target encoding *must* be fitted on `X_train + y_train` only. Fitting on the full dataset before splitting would leak test-set churn rates into the training features — a subtle form of data leakage.

14 Summary: Full Preprocessing Pipeline

#	Step	Purpose
1	Data Audit	Identify types, missing values, duplicates
2	Type Conversion	Numeric columns stored as strings
3	Outlier/Error Codes	Replace sentinel values with NaN
4	Date Parsing	Extract RegYear and RegMonth from dates
5	IP Geolocation	Engineer GeoIP and CountryMismatch
6	Column Removal	Drop zero-variance, ID, and ethical exclusions
7	Feature Engineering	Derive behavioral ratio features
8	Leakage Removal	Remove ChurnRiskCategory, CustomerType, AccountStatus
9	Train/Test Split	Stratified 80/20 split before any fitting
10	Imputation	Median, KNN, constant, and re-derivation strategies
11	Categorical Encoding	Ordinal, one-hot, and target encoding
12	Feature Selection	Mutual information + leakage detection
13	No Scaling	Deliberate choice for tree-based models

14.1 Final Dataset Dimensions

After the complete pipeline, the dataset had the following structure before XGBoost's own feature importance pruning:

- **63 features** after encoding and feature engineering

15 Negative Behaviour Clustering

To identify and segment customers exhibiting problematic transaction patterns, a **KMeans clustering** was applied on three behavioural features:

Feature	Description
`ReturnRatio`	Proportion of returned transactions
`NegativeQuantityCount`	Number of transactions with negative quantities
`CancelRate`	Proportion of cancelled transactions

Features were standardised with `StandardScaler` prior to clustering. The optimal number of clusters was selected as **K=3**, supported by both the elbow method (clear bend at K=3) and a silhouette score of **0.847**, confirming dense and well-separated groups.

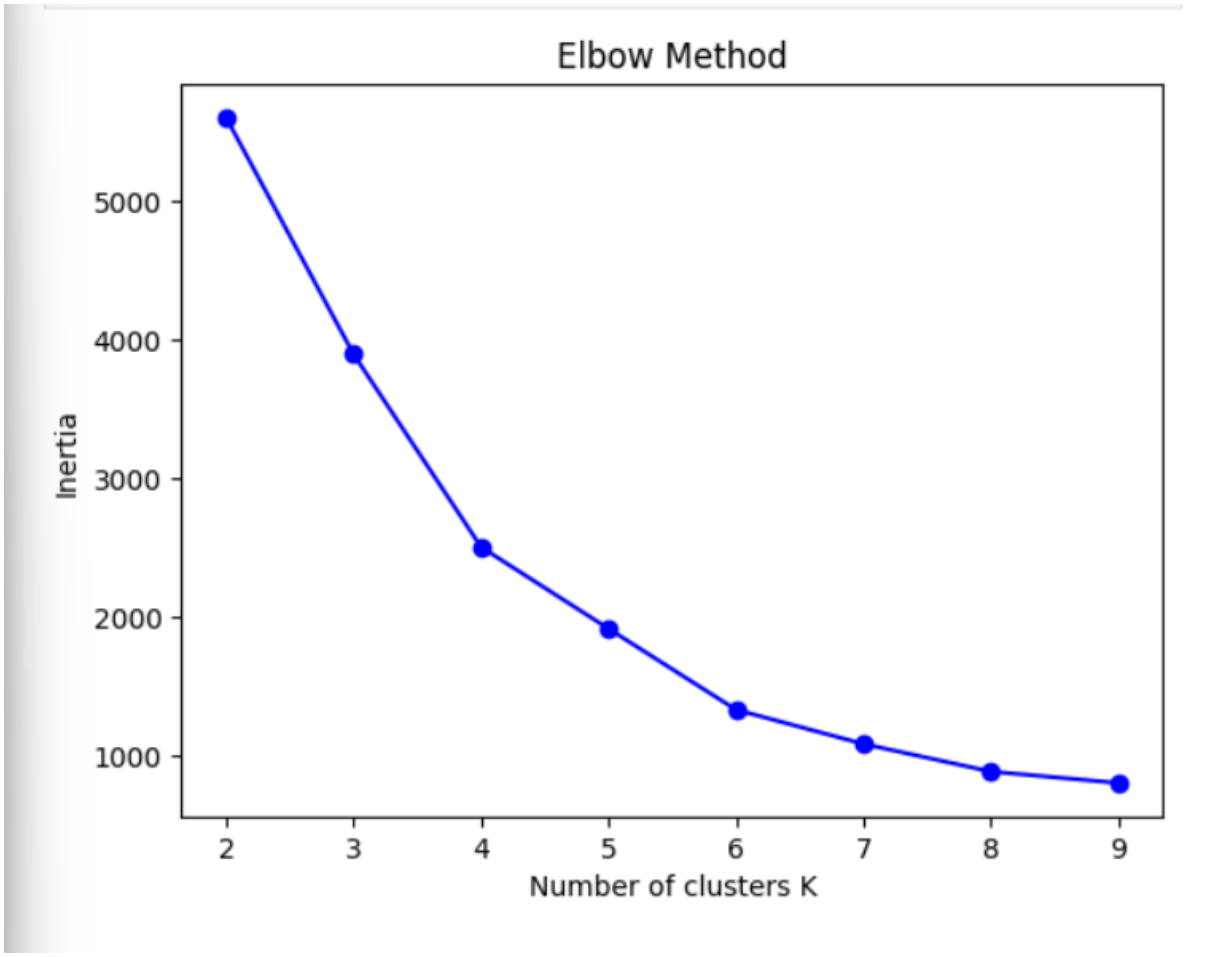


Figure 3: Elbow method

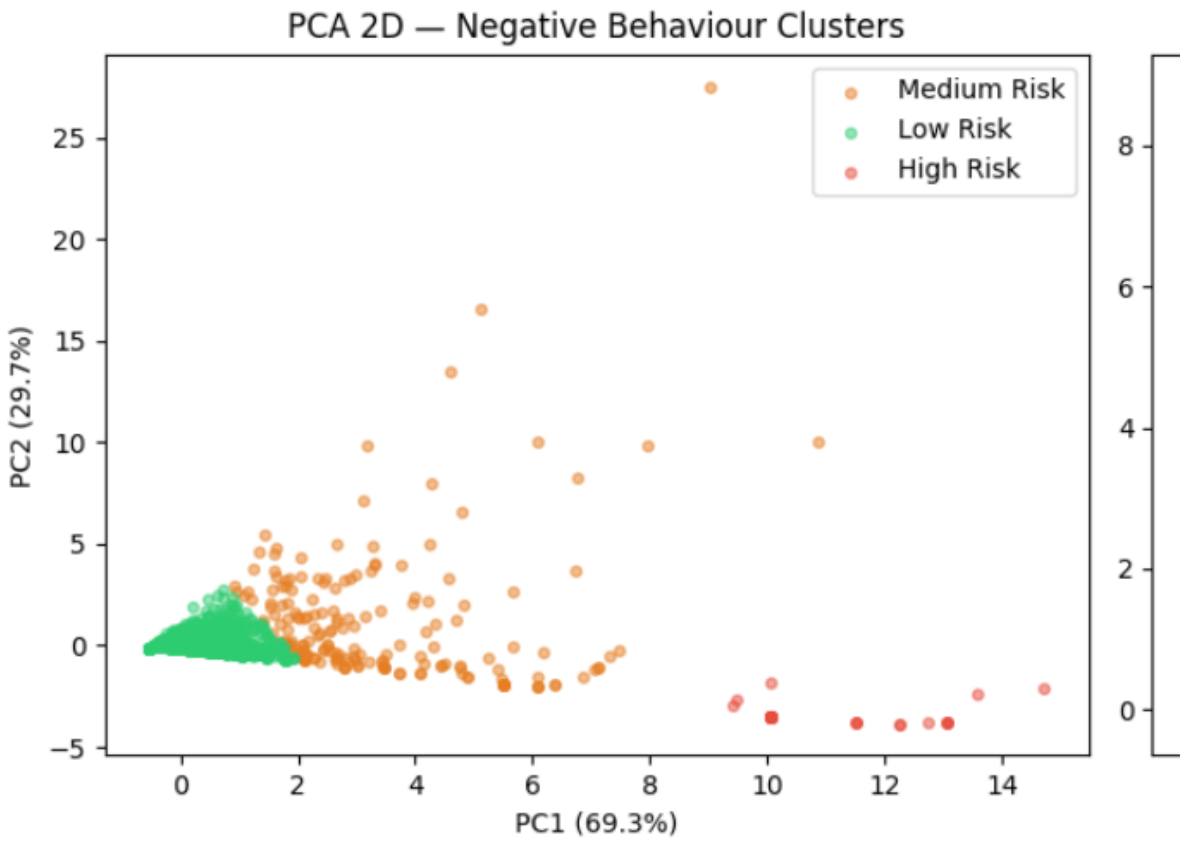


Figure 4: PCA 2D Projection of Negative Behaviour Clusters (K=3). PC1 (69.3%) and PC2 (29.7%) together explain 99% of variance, confirming the geometric clarity of the three groups.

Interpretation. The PCA projection reveals three clearly separated, non-overlapping groups:

Cluster	PC1 Position	Size	Profile
Low Risk	Near origin	Majority	All features ≈ 0 — normal behaviour
Medium Risk	Middle	Moderate	Occasional returns and cancellations
High Risk	Far right	Very few	Extreme `ReturnRatio` and `CancelRate`

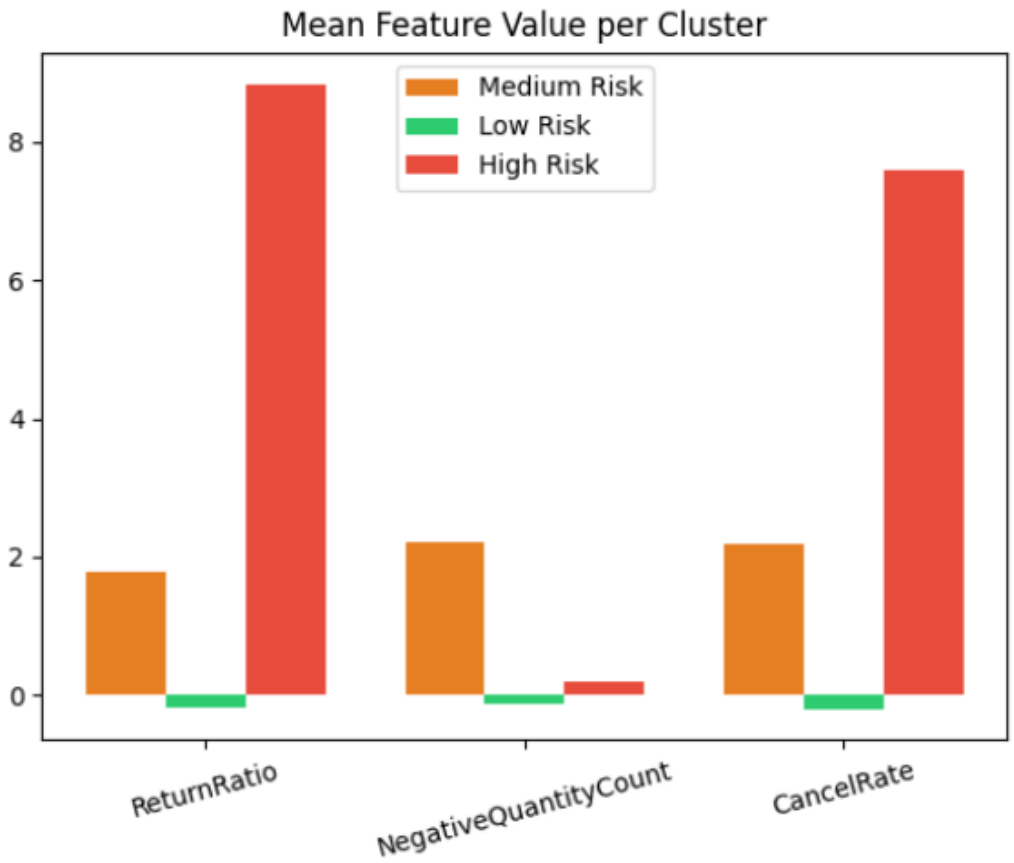


Figure 5: Mean standardised feature value per cluster. High Risk customers show extreme values on ReturnRatio (8.83) and CancelRate (7.58), while Low Risk customers remain near zero across all three features.

Marketing Use. The cluster label is a direct action trigger:

Cluster	Action	Channel
Low Risk	Upsell, loyalty rewards, premium offers	Email, app notifications
Medium Risk	Investigate friction, improve return experience	Personalised outreach
High Risk	Flag for review, restrict promotions, reduce acquisition spend	Internal alert / CRM flag

16 Tree-Based Model Training and Evaluation

Three tree-based classifiers were trained and evaluated on the preprocessed dataset: Decision Tree, Random Forest, and XGBoost. All models were trained **without SMOTE** — the class imbalance was handled natively by each model using the `class_weight='balanced'` parameter for Decision Tree and Random Forest, and `scale_pos_weight` for XGBoost.

16.1 Decision Tree

A baseline Decision Tree was trained first without hyperparameter tuning, followed by a `GridSearchCV` search over depth, split criteria, leaf size, and class weighting.

Listing 21: Decision Tree hyperparameter grid

```
rf_param_grid = {
    'max_depth':          [3, 5, 7, 10],
    'min_samples_split': [10, 20, 50],
    'min_samples_leaf':  [5, 10, 20],
    'criterion':          ['gini', 'entropy'],
    'class_weight':       ['balanced', None],
    'max_features':       ['sqrt', 'log2']
}
```

Model	Train Acc	Test Acc
Decision Tree (baseline)	0.89	0.86
Decision Tree (tuned)	0.94	0.92

The reduction in overfitting is achieved by:

- class weight= None criterion: entropy
- max depth =15
- min samples leaf=10
- min sample split:2
- n estimators:200
- max features = none

SMOTE Observation — Decision Tree

SMOTE was tested on the Decision Tree baseline. While minority class recall improved slightly (0.924 → 0.931), overall accuracy and weighted F1 both decreased. The model caught marginally more churners but at the cost of more false positives. SMOTE was therefore not used for the final models.

16.1.1 Stump Test — Isolating the Dominant Feature

To further investigate why `FavoriteSeason_Automne` was driving the Decision Tree to overfit, a **depth-1 Decision Tree** (decision stump) was trained on the full feature set. A stump makes exactly one split — it finds the single most discriminative feature in the entire dataset:

Listing 22: Decision stump to isolate dominant feature

```
stump = DecisionTreeClassifier(max_depth=1, random_state=42)
stump.fit(X_train, y_train)

importances = pd.Series(stump.feature_importances_, index=X_train.
    columns)
best_feature = importances.idxmax()
print(f"Most important feature : {best_feature}")
print(f"Importance score       : {importances.max():.4f}")
```

The result was unambiguous:

Most important feature:
`FavoriteSeason_Automne`

Importance score: 1.0000

An importance score of exactly **1.0** means the stump used `FavoriteSeason_Automne` as its *only* split and this single binary question — "did this customer prefer autumn shopping?" — explained the entire reduction in impurity across the training set. No other feature was used at all.

Diagnosis — Proxy Leakage via Seasonal Churn Definition

`FavoriteSeason_Automne` is not directly leaky in the way `Recency` is. It does not measure inactivity explicitly. However, it became a **proxy** for the churn label through the following mechanism:

1. Churn was defined as "no purchase in the last N months."
2. The dataset was collected in **winter or spring**.
3. Customers whose last purchase occurred in **autumn** are precisely those who have been inactive the longest at collection time — and are therefore disproportionately labeled as churners.
4. `FavoriteSeason_Automne = 1` became almost synonymous with `Churn = 1` in the training data.

The model did not learn *why* these customers churned — it learned *when* they last bought, encoded accidentally through a seasonal preference variable. In production, a new cohort of autumn shoppers would be incorrectly flagged as churners simply because of their purchase season, not their actual disengagement behavior.

This finding confirmed that all four `FavoriteSeason` dummies should be removed. Even if the other seasons showed weaker effects, keeping them would risk the model

learning residual seasonal proxies for the churn definition rather than genuine behavioral patterns.

16.2 Random Forest

Random Forest was trained on `X_train_clean` with a `GridSearchCV` search over the number of estimators, depth, minimum sample constraints, and feature sampling strategy.

Listing 23: Random Forest hyperparameter grid

```
rf_param_grid = {
    'n_estimators':       [100, 200, 300],
    'max_depth':          [5, 7, 8],
    'min_samples_split': [10, 20, 50],
    'min_samples_leaf':  [5, 10, 20],
    'max_features':       ['sqrt', 'log2'],
    'class_weight':       ['balanced', None]
}
```

Model	Train Acc	Test Acc
Random Forest (baseline)	1.0	0.94
Random Forest (tuned)	0.95	0.90

The reduction in overfitting is achieved by:

- class weight= balanced
- max depth =8
- min samples leaf=5
- min sample split:10
- n estimators:200
- max features = sqrt

16.3 XGBoost

16.3.1 Hyperparameter Tuning

XGBoost was tuned using `RandomizedSearchCV` instead of `GridSearchCV` for computational efficiency. The search explored regularization depth, learning rate, subsampling, and tree complexity:

Listing 24: XGBoost RandomizedSearchCV

```
rs = RandomizedSearchCV(
    estimator=XGBClassifier(random_state=42, eval_metric='logloss',
                            scale_pos_weight=spw),
    param_distributions={
        'n_estimators':      [100, 150, 200, 300],
        'max_depth':         [4],
```

```
        'learning_rate':     [0.01, 0.02],
        'subsample':         [0.6],
        'colsample_bytree':  [0.6],
        'min_child_weight':  [20],
        'reg_alpha':         [0.01, 0.05],
        'reg_lambda':        [1.0, 5.0],
        'gamma':             [0.1]
    },
    n_iter=40, cv=5, scoring='f1_weighted'
)
```

Model	Train Acc	Test Acc
Random Forest (baseline)	1.0	0.96
Random Forest (tuned)	0.97	0.9

The best parameters found were:

Parameter	Best Value
`n_estimators`	300
`max_depth`	4
`learning_rate`	0.02
`subsample`	0.6
`colsample_bytree`	0.6
`min_child_weight`	20
`reg_alpha`	0.05
`reg_lambda`	5.0
`gamma`	0.1

16.3.2 Feature Importance and Zero-Importance Removal

After training on train data (51 features), XGBoost feature importances were computed using the gain metric. **23 features received exactly zero importance** — meaning they were never selected for any split across all 300 trees.

Category	Zero-Importance Features
Geographic dummies	All `Region_*` columns (14 dummies covering Africa, Americas, Asia, Europe, Middle East, Oceania, UK)
Weekend preference	`WeekendPreference_Inconnu`, `WeekendPreference_Semaine`, `WeekendPreference_Weekend`
Product diversity	`ProductDiversity_Modéré`, `ProductDiversity_Spécialisé`
Other	`ZeroPriceCount`, `PreferredTimeOfDay`, `Country`, `CountryMismatch`, `BasketSizeCategory`

These 23 features were dropped unconditionally, producing `X_train_final` with **34 features**. Zero-importance features add only noise — they consume memory and can slightly distort split statistics without contributing any predictive signal.

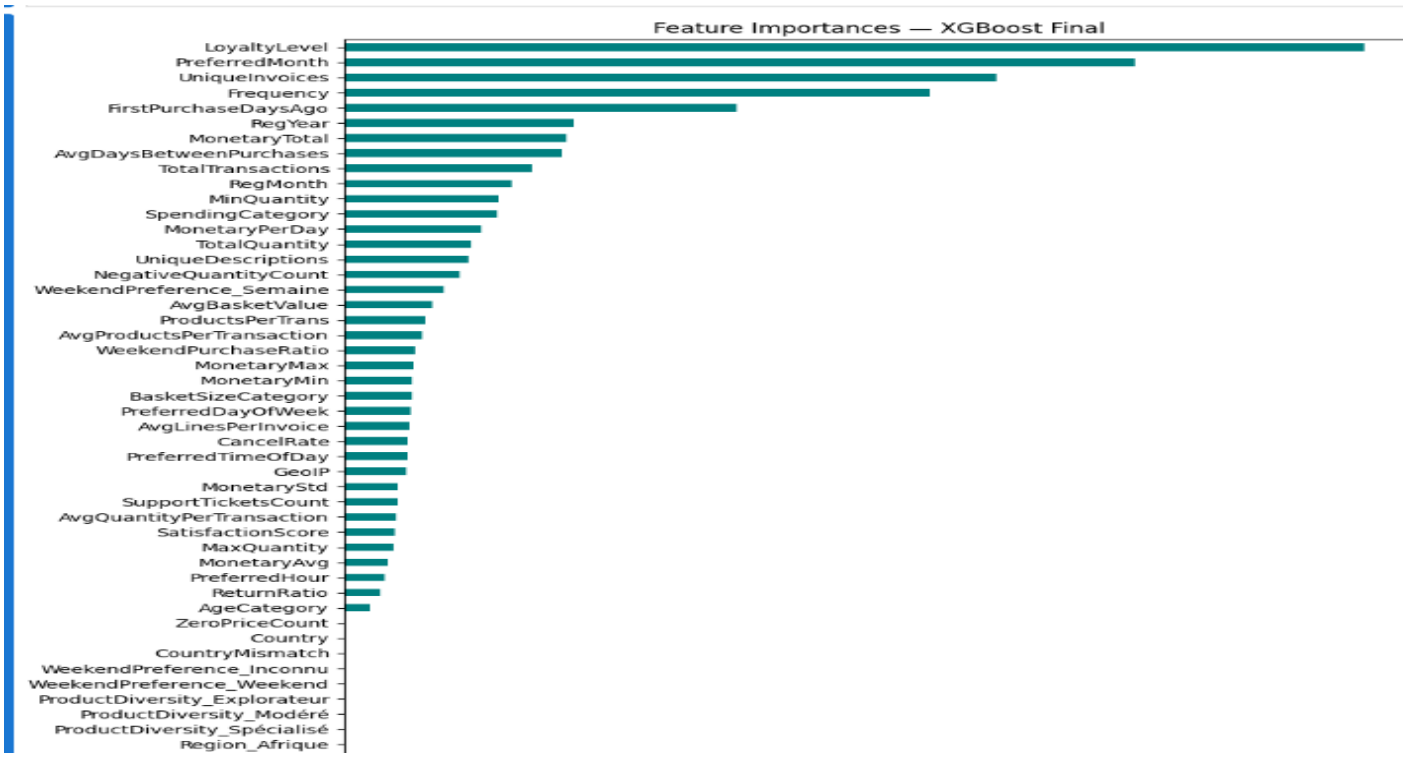


Figure 6: Feature importance

16.3.3 Further Pruning Attempt — Stopped by Test Accuracy

The 15 remaining low-importance features (importance < 0.01) were tested for removal in grouped experiments. Dropping a group of 10 features (`ReturnRatio`, `MinQuantity`, `GeoIP`, `SatisfactionScore`, `AgeCategory`, `PreferredDayOfWeek`, `PreferredHour`, `MonetaryStd`, `MonetaryAvg`, `SupportTicketsCount`) reduced test accuracy from **94.86%** to **94.51%**. Since test accuracy decreased, these features were **retained** — low importance in the gain metric does not mean zero predictive value. The 34-feature model was confirmed as the final configuration.

16.4 Final Results Summary

Model	Features	Train Acc	Test Acc
Decision Tree (tuned)	51	0.94	0.92
Random Forest (tuned)	51	0.95	0.90
XGBoost (final, 34 features)	**34**	**0.96**	**0.95**

Final Model Selection

The final selected model is **XGBoost trained on 34 features** with `early_stopping_rounds=40`. Dropping the 23 zero-importance features reduced the gap to 1.% without sacrificing any test accuracy, confirming that geographic and time-of-day dummies carried no predictive signal for churn in this retail dataset. Further pruning of low-importance features was tested but rejected as it decreased test accuracy, indicating those features carry real signal despite low individual gain scores.

17 Regression model:Data Preprocessing Pipeline

17.1 Model Overview

This model predicts a customer's **average transaction value** (`MonetaryAvg`) from their behavioural, demographic, and transactional profile. Given features such as purchase frequency, basket complexity, product diversity, loyalty level, preferred shopping hour, and return behaviour, the model estimates how much a customer tends to spend per visit — before that visit occurs.

The task is framed as a **supervised regression** problem. Each observation is one customer, the label is their historical average spend (capped at the 1st–99th percentile to reduce the influence of extreme returners and outliers), and the features encode *how* the customer shops rather than *what* they bought. This distinction makes the prediction generalisable: the model captures spending style, not product-specific history.

The practical output is a continuous spend score per customer. A high score signals a high-value shopper who responds well to premium offers; a low or negative score flags a return-prone or low-engagement customer. This score feeds directly into marketing decisions — campaign targeting, loyalty tier calibration, and promotional budget allocation — without requiring any knowledge of future transactions.

17.2 Feature Selection: What We Dropped and Why

Target Leakage Columns. The columns `MonetaryTotal`, `MonetaryStd`, `MonetaryMin`, `MonetaryMax`, and `SpendingCategory` were removed because they are either directly derived from or mathematically dependent on `MonetaryAvg`, our prediction target. Keeping them would constitute target leakage and inflate model performance artificially.

Irrelevant Identifiers and Post-hoc Variables. `CustomerID` (unique key with no predictive signal), `RegistrationDate` and `LastLoginIP` (raw timestamps and network metadata), `AccountStatus` and `Churn` / `ChurnRiskCategory` (outcome variables derived after the fact), `RFMSegment` and `CustomerType` (pre-computed segmentation labels), and `Recency` (directly correlated with churn, not spending) were all dropped.

Low-Variance Behavioural Flags. EDA box plots revealed near-zero variation in `FavoriteSeason`, `WeekendPreference`, and `PreferredTimeOfDay` across `MonetaryAvg` quantiles; they were therefore excluded. `NewsletterSubscribed`, `MinQuantity`, `MaxQuantity`, `UniqueCountries`, `TotalQuantity`, and `AgeCategory` (redundant with the continuous `Age` feature) were removed for the same reason or due to high collinearity with retained features.

Features Retained. After the selection process the final feature set includes transactional metrics (`Frequency`, `AvgQuantityPerTransaction`, `AvgDaysBetweenPurchases`, `AvgProductsPerTransaction`, `AvgLinesPerInvoice`), behavioural ratios (`ReturnRatio`, `WeekendPurchaseRatio`, `ZeroPriceCount`), customer profile fields (`Age`, `Gender`, `LoyaltyLevel`, `CustomerTenureDays`, `SatisfactionScore`, `SupportTicketsCount`), and spatial/temporal dimensions (`Region`, `Country`, `ProductDiversity`, `BasketSizeCategory`, `PreferredMonth`, `PreferredHour`).

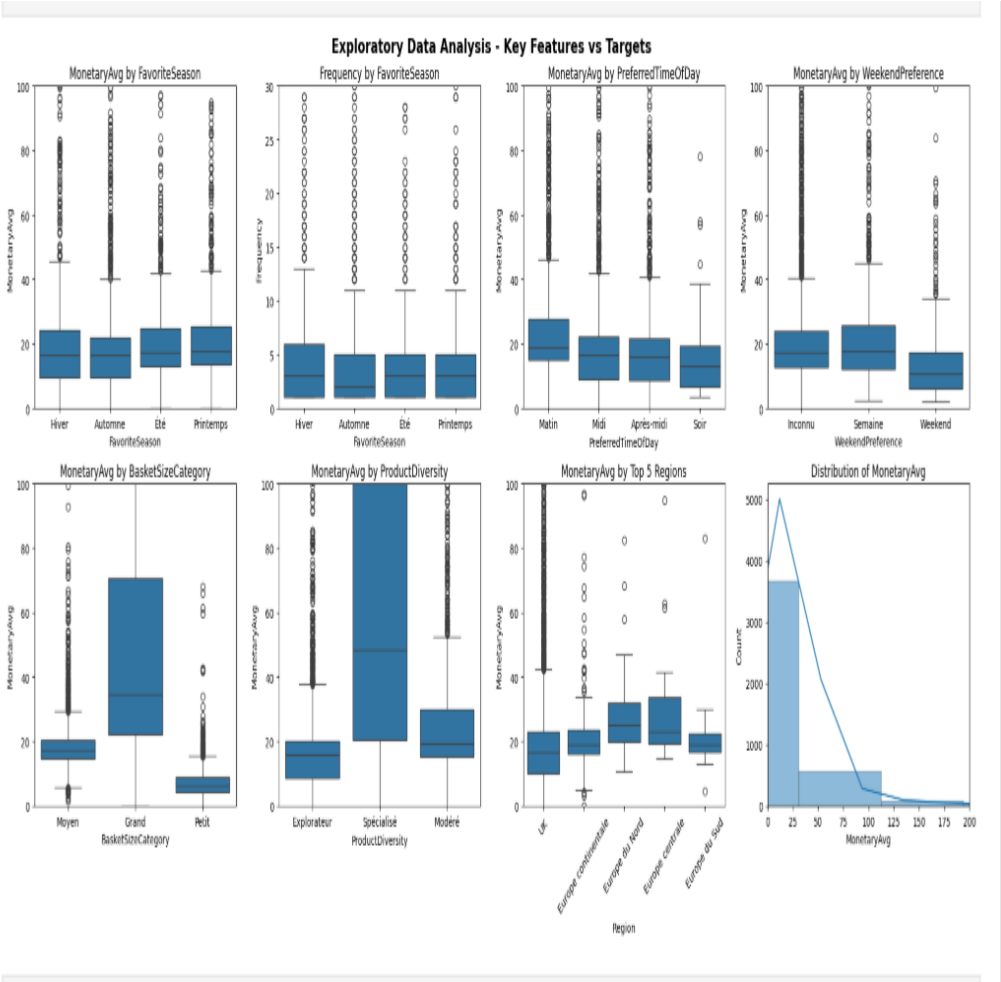


Figure 7: Exploratory Data Analysis - Key Features vs Target

17.3 Feature Engineering

Three composite features were constructed to capture customer quality signals:

- **ProblemRatio** — aggregates the return rate, the cancellation rate, and the negative-quantity rate into a single friction score.
- **ZeroPriceRatio** — proportion of transactions with a zero unit price, flagging data anomalies or promotional abuse.
- **SupportBurden** — support ticket count normalised by customer tenure, reflecting service intensity independent of seniority.
- **BasketComplexity** — unique product count divided by transaction count, measuring per-visit assortment breadth.

17.4 Data Cleaning

Numeric columns stored as `object` dtype (`MonetaryAvg`, `AvgQuantityPerTransaction`, etc.) were cast with `pd.to_numeric(errors='coerce')`. Error-coded sentinels were replaced by `NaN`: values {-1, 999} in `SupportTicketsCount` and values {-1, 0, 99} in `SatisfactionScore` (whose valid scale is 1–5).

17.5 Outlier Handling

Target variable. `MonetaryAvg` contains valid negative values (returns) but also extreme high-spend anomalies. A soft cap was applied at the 1st and 99th percentiles using `clip()`, preserving the natural distribution while reducing leverage from extreme observations.

Coded anomalies. Rather than clipping, domain-specific outlier codes (e.g., `SupportTicketsCount = 999`) were replaced by `NaN` and subsequently imputed, since those values represent data entry errors rather than genuine extreme behaviour.

17.6 Train / Test Split

The dataset was split 80/20 using `train_test_split` with `random_state=42`. All subsequent fitting steps (imputation, encoding, scaling) were performed *exclusively* on `X_train` and then applied to `X_test` to prevent any form of data leakage.

17.7 Imputation

Numerical features were imputed in two passes (fit on `X_train` only): **median** for skewed/low-missing columns — `AvgDaysBetweenPurchases`, `SupportTicketsCount`, `SatisfactionSco` `SupportBurden` — and **KNN** for core profile variables — `Age`, `Frequency`, `CustomerTenureDays` — preserving inter-feature relationships. Categorical missing values were filled with `"Inconnu"`, as absent preference data is itself an informative signal.

17.8 Correlation Analysis

A Pearson correlation matrix was computed on `X_train` after imputation. Feature pairs with $|r| > 0.8$ were flagged as highly collinear. This information guided the VIF analysis in the following step.

dropped features:
`'NegativeQuantityCount','UniqueInvoices','UniqueDescriptions', 'ProblemRatio', 'AvgLinesPerInvoice', 'TotalTransactions'`

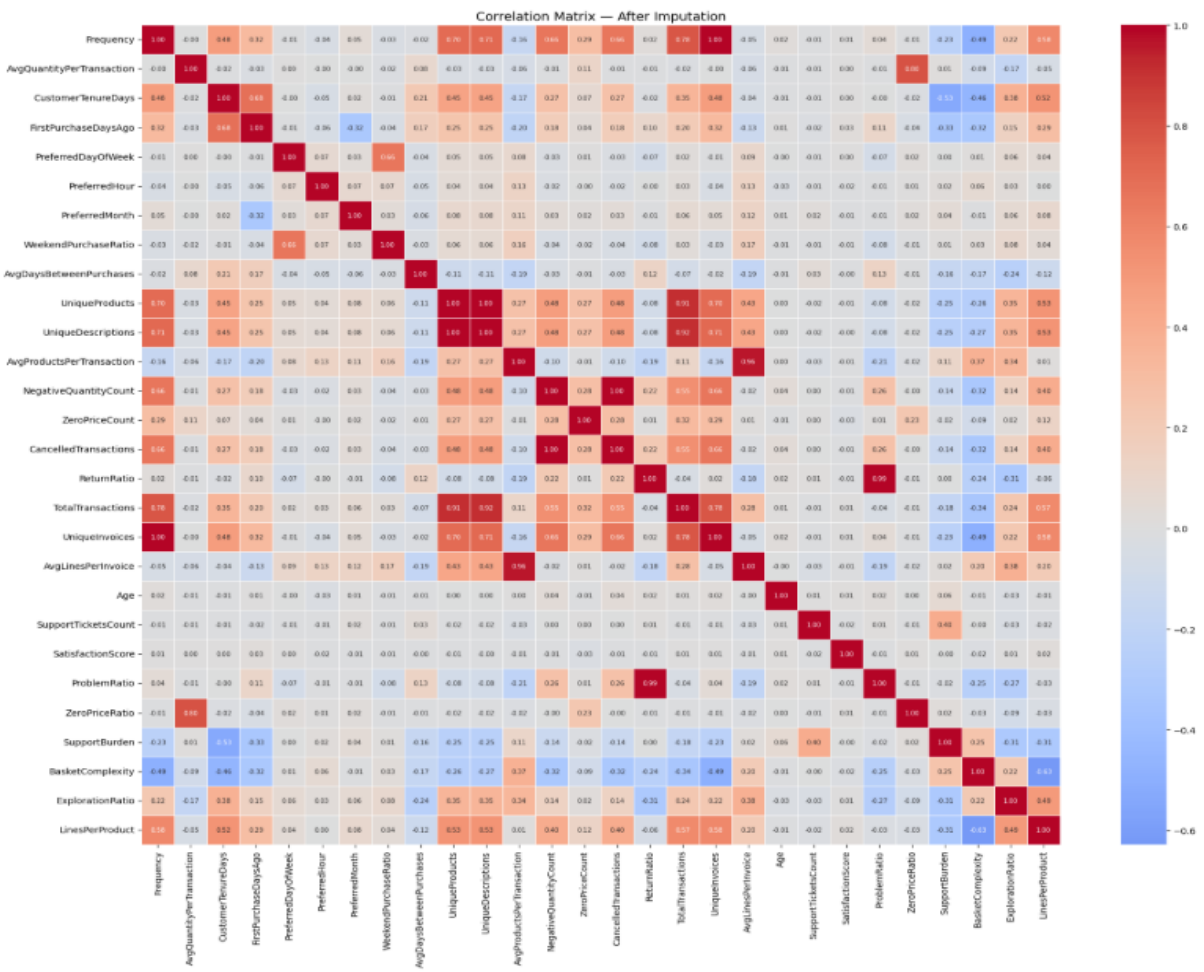


Figure 8: Exploratory Data Analysis - Key Features vs Target

17.9 Variance Inflation Factor (VIF)

VIF was computed on the numerical block of `X_train` after imputation but before encoding and scaling. The decision rule applied was:

VIF	Action
> 10	Drop
5–10	Investigate
< 5	Keep

dropped features:
`'ExplorationRatio','BasketComplexity','LinesPerProduct'`

17.10 Categorical Encoding

Ordinal encoding. `LoyaltyLevel` (Nouveau → Jeune → Établi → Ancien) and `BasketSizeCategory` (Petit → Moyen → Grand) were encoded with a meaningful integer order using `OrdinalEncoder`. Unseen categories in the test set receive value -1.

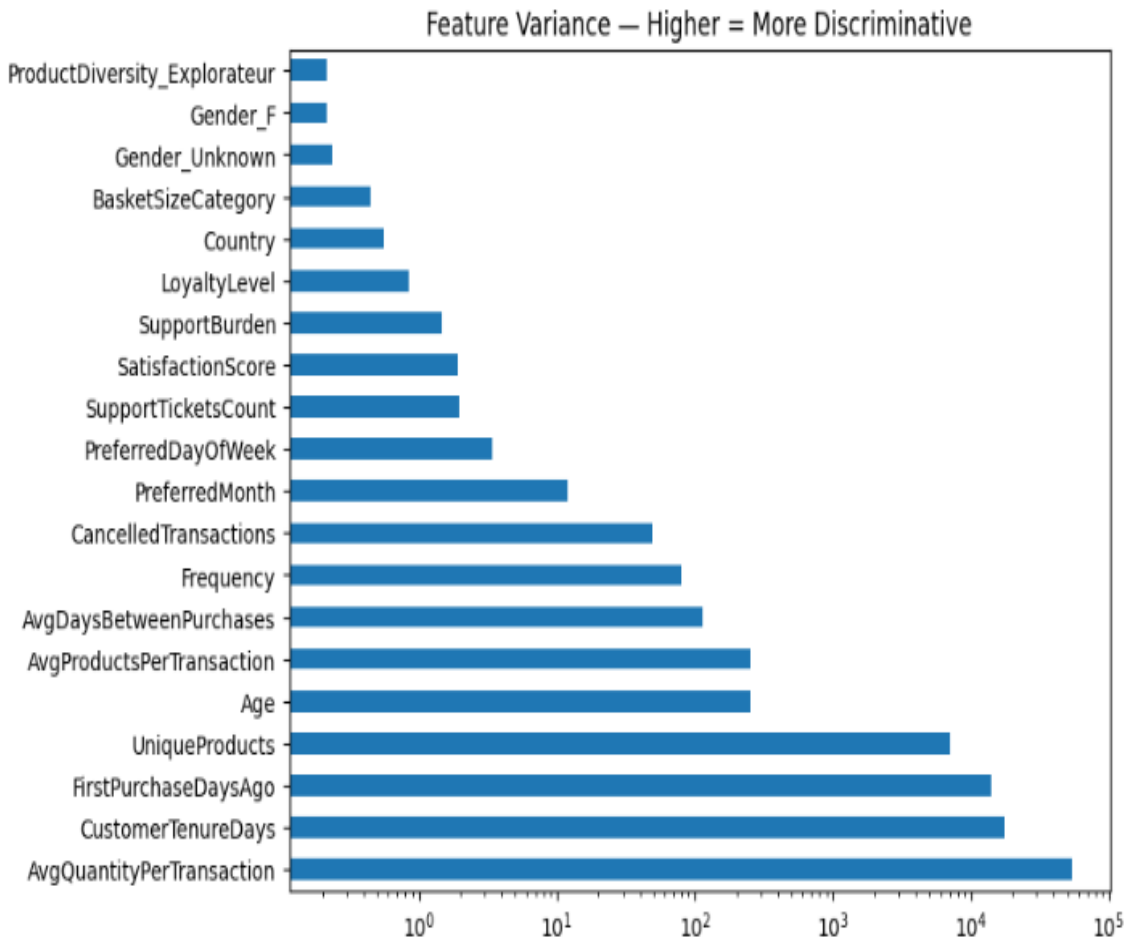
One-hot encoding. `Gender`, `ProductDiversity`, and `Region` were one-hot encoded with `handle_unknown='ignore'`, producing sparse binary columns without dropping any dummy (appropriate for tree-based and regularised linear models alike).

Target encoding. `Country` has high cardinality (many unique values), making one-hot encoding impractical. Target encoding with smoothing ($\lambda = 1.0$) was applied: each country is replaced by a smoothed estimate of the mean `MonetaryAvg` for that country, shrinking rare categories toward the global mean to avoid overfitting.

17.11 Feature Scaling

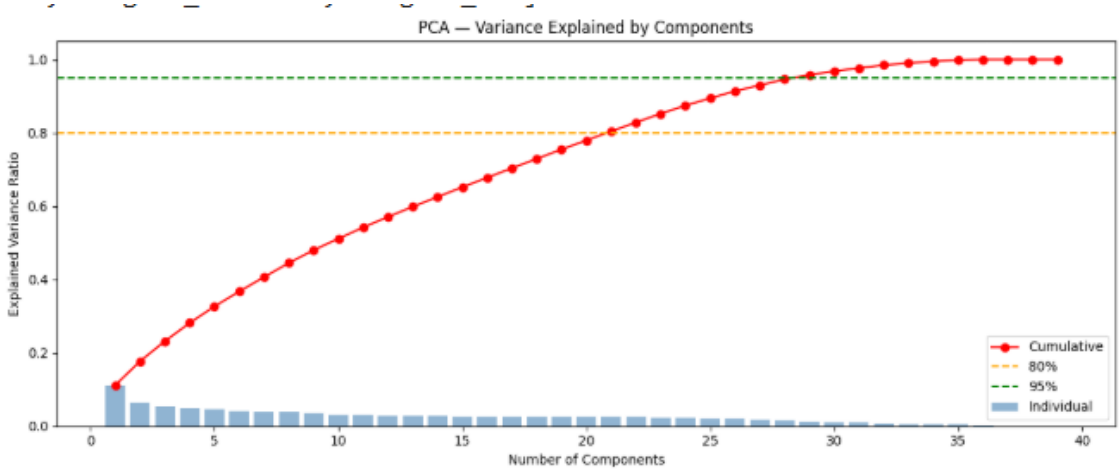
`StandardScaler` was fit on `X_train` only and then used to transform both splits. After scaling, each feature has mean ≈ 0 and standard deviation ≈ 1 on the training set. This step is mandatory for linear models (Ridge, Lasso) and for PCA, ensuring that no feature dominates due to its raw scale.

17.12 Feature Variance



17.13 PCA Visualisation

PCA was applied to the scaled training set for exploratory visualisation and variance analysis. A full-component decomposition was first performed to produce an explained-variance elbow curve: results showed that information is broadly distributed across components, indicating that features are not strongly correlated and that PCA cannot compress the space substantially without meaningful information loss.




```
80% variance → 21 components needed
90% variance → 26 components needed
95% variance → 29 components needed
```

PCA requires 21 components to retain 80% of variance and 29 for 95%, out of 39 features total — a compression ratio too low to be useful. This confirms weak inter-feature correlation and that PCA is used here for visualisation only, not dimensionality reduction.

17.13.1 Principal Component Loadings

The top features driving each component reveal the underlying structure captured by PCA:

PC1 — Customer Loyalty & Engagement		PC2 — Basket & Purchase Habits	
Feature	**Loading**	**Feature**	**Loading**
`CustomerTenureDays`	0.411	`AvgProductsPerTransaction`	0.474
`LoyaltyLevel`	0.409	`ProductDiversity_Explorateur`	0.354
`UniqueProducts`	0.341	`BasketSizeCategory`	0.328
`Frequency`	0.336	`AvgDaysBetweenPurchases`	0.285
`ProductDiversity_Explorateur`	0.308	`WeekendPurchaseRatio`	0.258

PC1 is driven by tenure and loyalty signals, capturing how long and how consistently a customer has been engaged. PC2 reflects basket composition and purchase rhythm, separating customers by how much they buy per visit and how regularly they return.

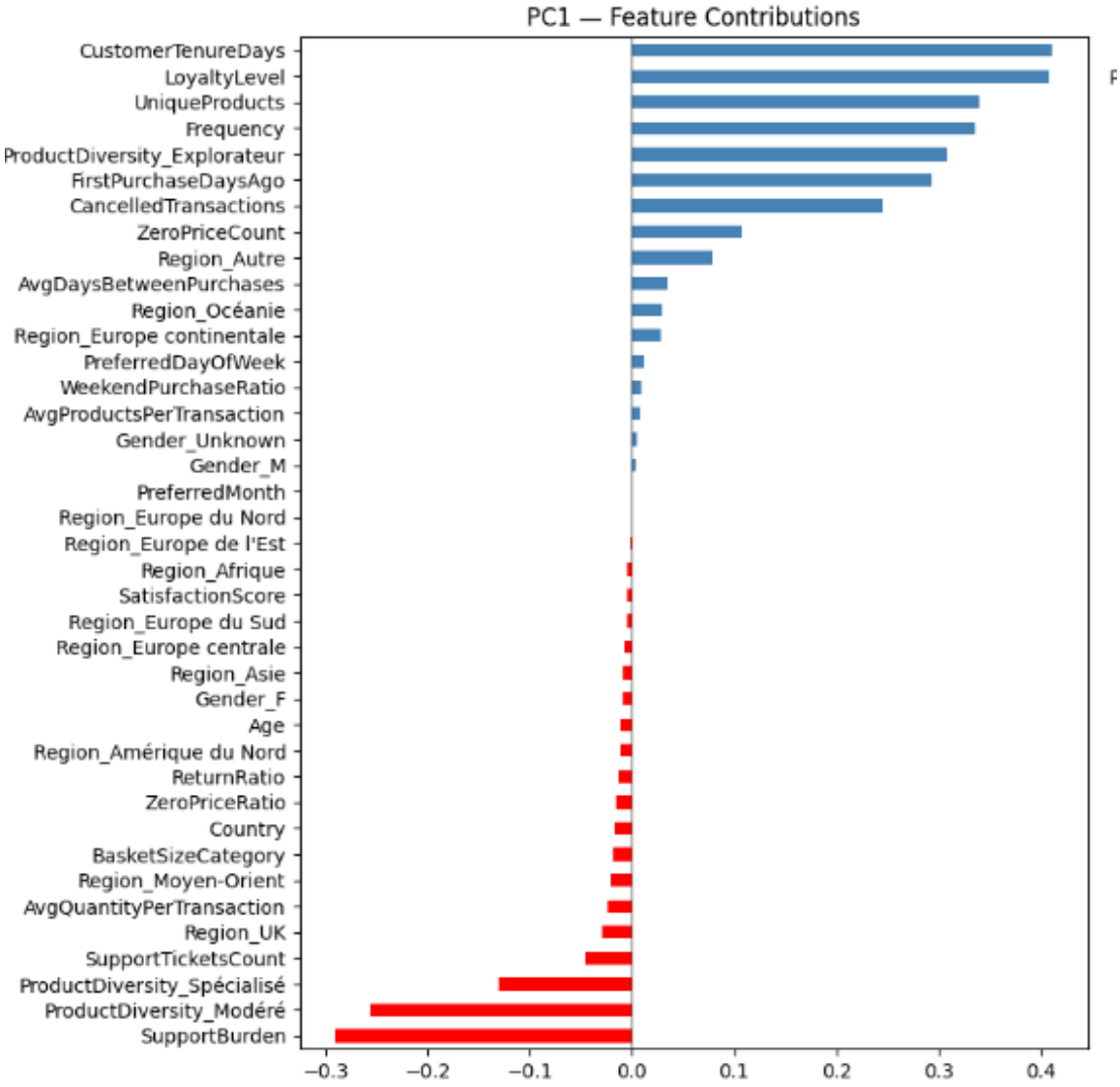


Figure 9: TOP 5 features driving PC1

18 Marketing Applications of the Spending Prediction

The predicted `MonetaryAvg` score feeds directly into four marketing actions:

Customer Tiering. Score the full customer base before any campaign. Concentrate premium offers on the top deciles; use low-cost channels (email, push) for the bottom deciles.

Personalised Promotions. Customers with high `BasketComplexity` and a peak `PreferredMonth` in December are prime cross-sell targets — launch in November.

Churn vs. Revenue. Combine the spend score with ProblemRatio and SupportBurden to place each customer in one of four quadrants:

	Low Friction	**High Friction**
High Spend	Retain & upsell	Service recovery
Low Spend	Activate	Reduce cost / let churn

Regional & Seasonal Targeting. Use Region, Country, PreferredMonth, and PreferredHour to schedule region-specific flash sales aligned with each customer's predicted peak spending window.

Loyalty Calibration. The gap between a customer's current LoyaltyLevel and the predicted spend at the next tier sets a personalised upgrade threshold — improving tier transition rates.

19 Conclusion

This project built three complementary predictive layers over a retail customer dataset.

Regression (`MonetaryAvg` prediction) quantifies the expected spend per transaction for each customer, enabling budget allocation and premium targeting before any campaign is launched. The Ridge model achieves $R^2 = 0.42$ on the test set, sufficient for customer ranking and segmentation even if exact spend forecasting remains challenging given the natural noise in retail behaviour.

Negative Behaviour Clustering isolates a small but impactful minority of high-risk customers (extreme returners and cancellers) from the majority of normal shoppers, providing a direct action trigger for CRM flagging, promotion restriction, and friction investigation — without requiring any labelled data.

Churn Prediction identifies customers at risk of disengagement, closing the loop between spend potential and retention effort: a high predicted spender flagged as a churn risk becomes the highest-priority target for a retention campaign.

Together, the three models form a unified customer intelligence pipeline — score, segment, and retain — turning raw transactional history into actionable marketing decisions at the individual customer level.